Lab 3

MSDS684_S70_Reinforcement Learning

Maryi Tatiana Palacios Giraldo

Regis University

March 2026

Project Overview

This lab investigates the application of Monte Carlo (MC) methods for reinforcement learning, focusing on first-visit Monte Carlo control using ε-soft policies. Unlike Dynamic Programming methods explored in previous labs, Monte Carlo methods do not require knowledge of the environment's transition dynamics. Instead, they learn directly from experience by averaging returns obtained from complete episodes (Sutton & Barto, Chapter 5).

The objective of this lab is to implement an on-policy Monte Carlo control algorithm in the Blackjack environment provided by Gymnasium. Blackjack represents a finite Markov Decision Process where the agent must decide whether to "hit" or "stick" based on the current state. The state space consists of the player's current sum, the dealer's visible card, and whether the player holds a usable ace. The action space is discrete, consisting of two possible actions: hit or stick. Rewards are sparse and episodic, with +1 for a win, 0 for a draw, and -1 for a loss. Episodes terminate when the player busts, sticks and the dealer finishes playing, or the game otherwise reaches a terminal condition.

This lab focuses on learning optimal policies through experience without a model of the environment. The first-visit Monte Carlo approach updates the action-value function $Q(s,a)$Q(s,a) only on the first occurrence of each state-action pair within an episode, ensuring unbiased estimates. Policy improvement is achieved through an ε-soft strategy, which balances exploration and exploitation by assigning a small probability to all actions while favoring the greedy action.

It is expected that the agent will gradually improve its policy by observing returns from many episodes, converging toward a near-optimal Blackjack strategy. However, due to the stochastic nature of the environment and the high variance of Monte Carlo estimates, convergence may be slow and noisy. Additionally, maintaining exploration through ε-soft policies is expected to introduce variability in performance while preventing premature convergence to suboptimal strategies.

Overall, this lab aims to demonstrate how reinforcement learning agents can learn effective decision-making strategies purely from sampled experience, highlighting both the strengths and limitations of Monte Carlo methods.

Deliverables

GitHub Repository:

https://github.com/MPalacios-Analytics/DataScience_Projects/tree/d4aa79754c07ec2ff0b6dcdde78dd26e08186d74/06_Reinforcement_Learning/Lab_3

Implementation Summary

The implementation uses first-visit Monte Carlo control with an ε-soft policy in the Blackjack environment from Gymnasium. Episodes are generated by interacting with the environment using env.reset() and env.step() until termination. Each episode is stored as a sequence of (state, action, reward) tuples.

Returns are computed by iterating backward through each episode, and only the first occurrence of each state-action pair is used to update the action-value function $Q(s, a)$. The policy is improved incrementally using an ε-greedy strategy derived from the current Q-values.

The agent was trained for 500,000 episodes with ε values of 0.1 and 0.01 to evaluate the impact of exploration. Performance was evaluated using value function visualizations and learning curves with smoothing applied to average episode rewards.

Key Results & Analysis

The learned value function and policy demonstrate that the agent successfully captures key aspects of optimal Blackjack strategy. States with higher player sums (e.g., 17 or above) tend to favor the "stick" action, while lower sums favor "hit," particularly when the dealer shows a strong card. This behavior aligns with standard Blackjack strategy and reflects the agent's ability to learn effective decision rules through experience.

The value function plots reveal that states with a usable ace generally have higher expected returns compared to those without one. This is because the ace provides flexibility, allowing the agent to take more aggressive actions with reduced risk of busting. As a result, the agent assigns higher values to these states, consistent with theoretical expectations.

The learning curve shows a clear improvement in performance during the early stages of training, particularly within the first 50,000 episodes. During this phase, the agent collects sufficient experience to estimate $Q(s, a)$for frequently visited states. After approximately 100,000 episodes, the learning curve begins to plateau, indicating that the policy has largely stabilized.

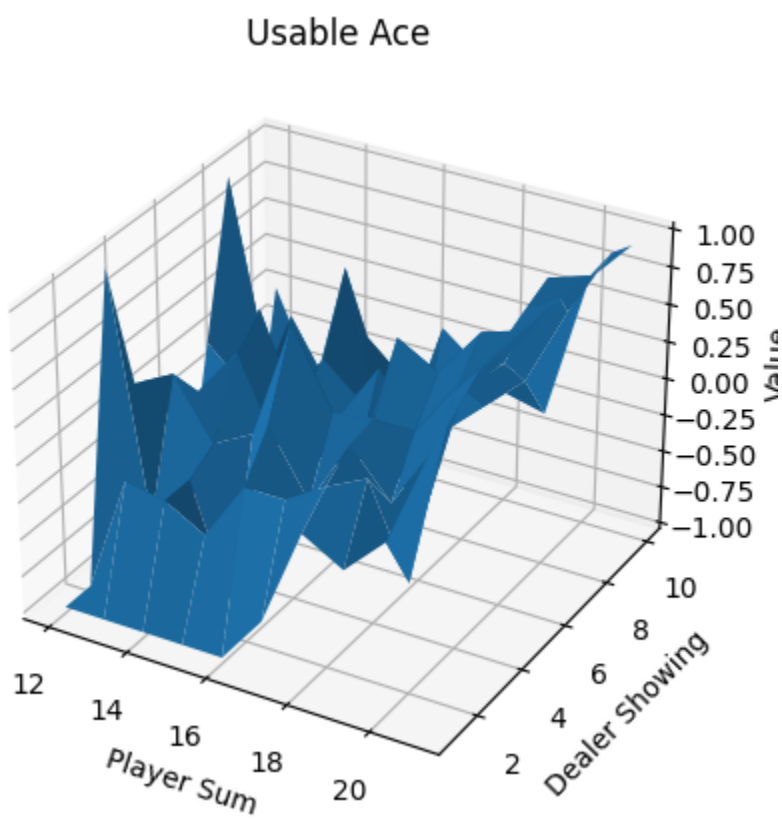
Despite smoothing, the curve exhibits noticeable fluctuations, which can be attributed to the high variance inherent in Monte Carlo methods. Since updates are based on complete episodes and not bootstrapped estimates, the agent's performance continues to oscillate even after convergence. This behavior is consistent with Monte Carlo theory (Sutton & Barto, Chapter 5).

The comparison between ε = 0.1 and ε = 0.01 highlights the exploration–exploitation tradeoff. A higher ε value leads to increased exploration, allowing the agent to discover higher-value strategies but resulting in greater variability. In contrast, a lower ε produces more stable performance but may limit the agent's ability to improve further due to reduced exploration.

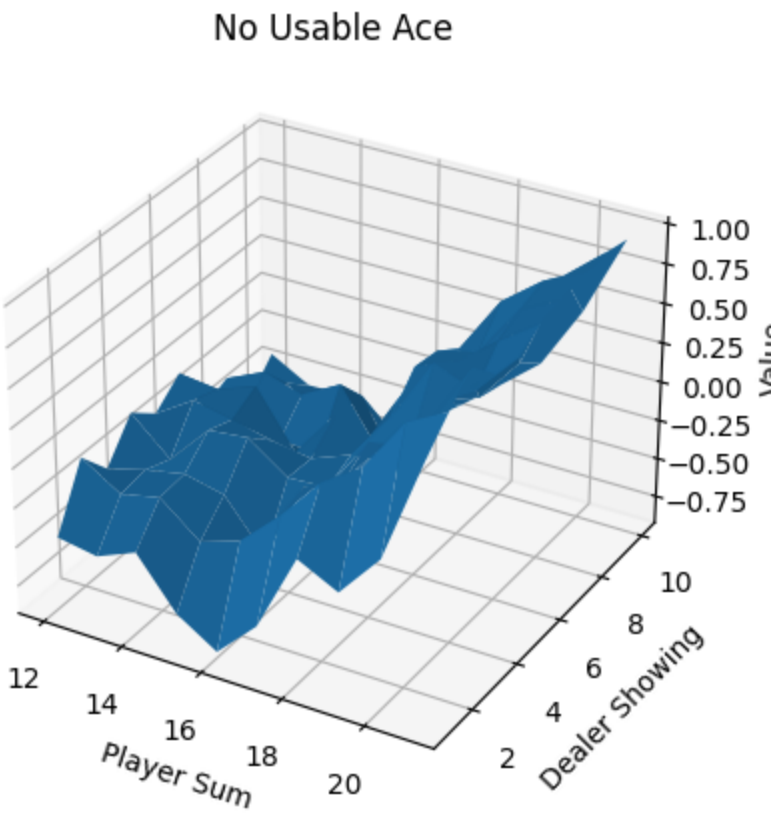
Overall, the results demonstrate that Monte Carlo control can effectively learn near-optimal policies from experience, though at the cost of high variance and relatively slow convergence compared to other reinforcement learning methods.

Visualizations

Value Function (usable ace)

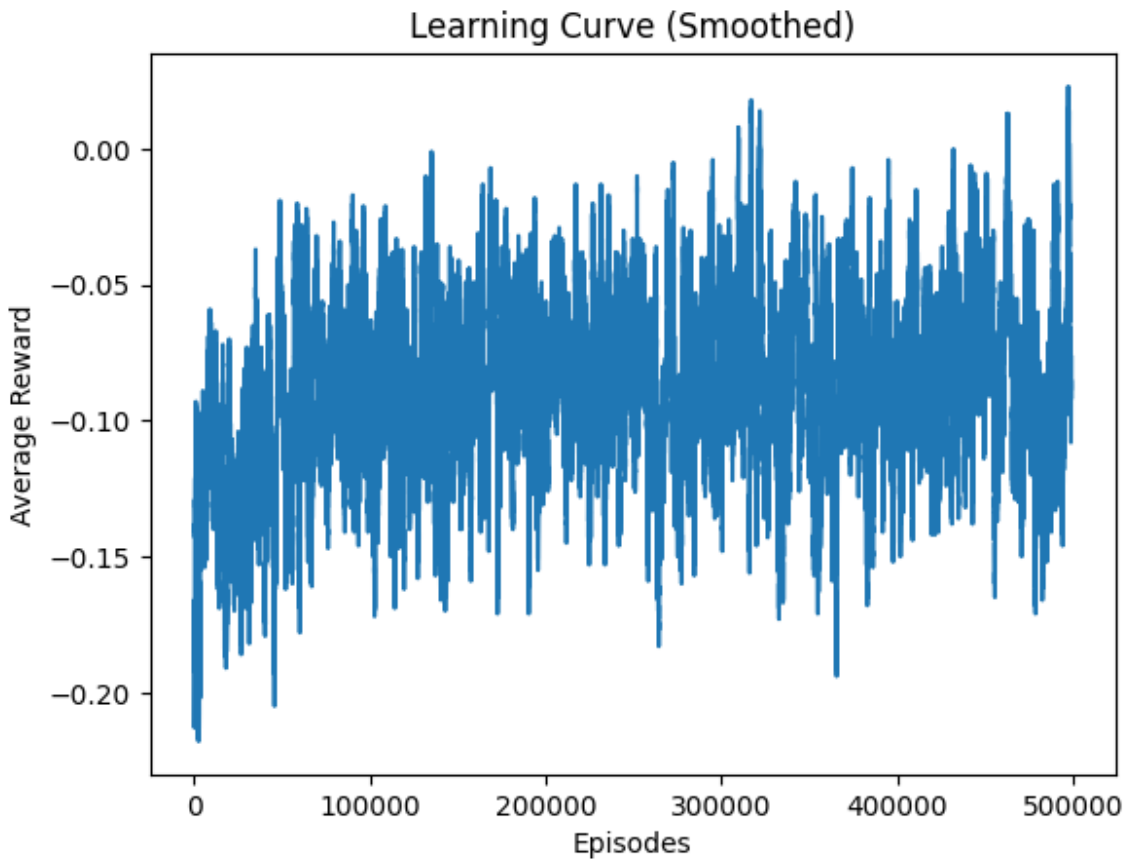


Value Function (no usable ace)



The value function with a usable ace shows higher values across a wider range of player sums, reflecting the increased flexibility provided by the ace. Since the ace can be counted as 1 or 11, the agent can take more aggressive actions with lower risk of busting, resulting in higher expected returns. This behavior is consistent with Monte Carlo estimates derived from complete episodes (Sutton & Barto, Chapter 5).

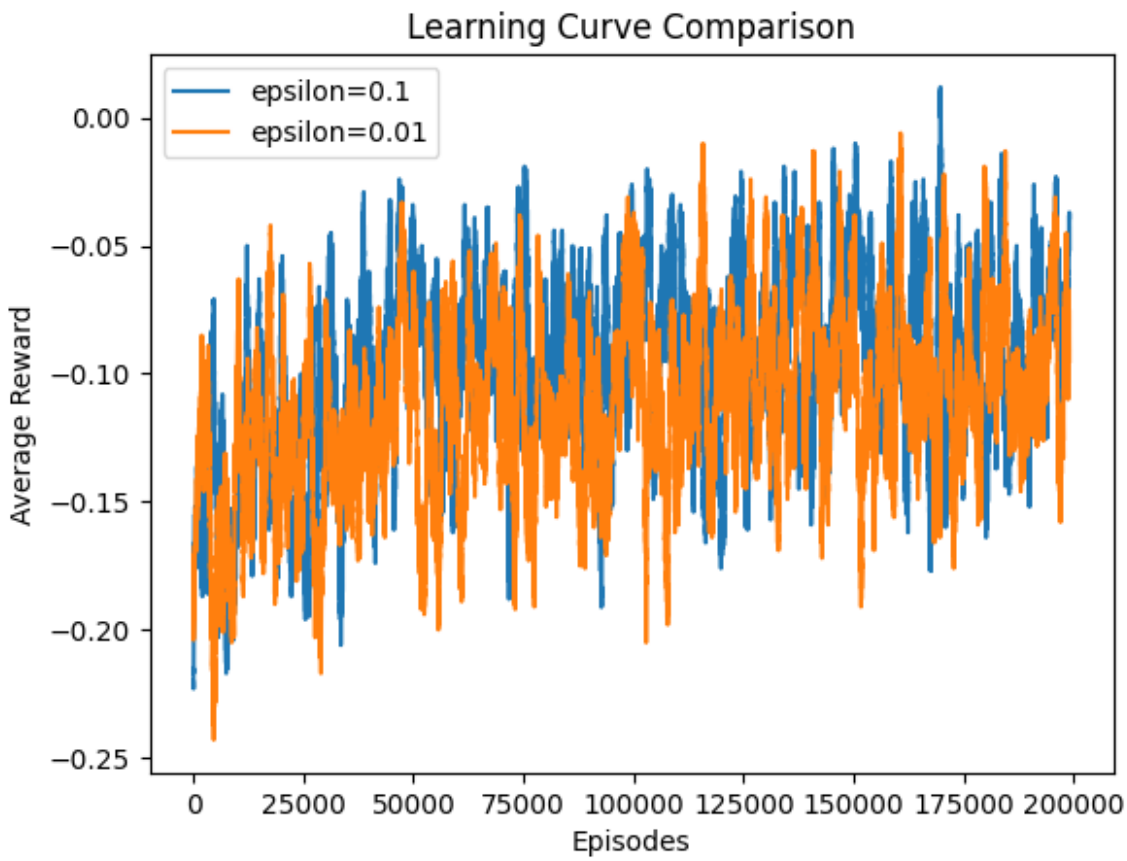
Learning Curve



From 0 to 100,000 episodes, the agent is collecting sufficient samples to estimate the action-value function $Q(s,a)$Q(s,a) for frequently visited states. During this phase, there is clear learning as the agent improves its estimates based on observed returns from complete episodes.

From 100,000 to 500,000 episodes, the agent enters a near-stationary regime where the policy has largely stabilized. The updates, based on full-episode returns, continue to fluctuate around a mean due to the high variance inherent in Monte Carlo methods. Since the policy is already close to optimal, further improvements are limited, and the agent primarily refines its estimates rather than discovering significantly better strategies.

ε comparison



The comparison between ε = 0.1 and ε = 0.01 highlights the exploration–exploitation tradeoff in Monte Carlo control. With ε = 0.1, the agent explores more frequently, which allows it to discover higher-value state-action pairs, reflected in occasional peaks in performance. However, this increased exploration also leads to higher variance, as the agent continues to take suboptimal actions with non-negligible probability.

In contrast, ε = 0.01 results in a more stable learning curve with lower variance. The agent exploits its current policy more consistently, leading to smoother performance over time. However, the reduced exploration limits its ability to discover potentially better strategies, which may explain the slightly lower peak performance compared to ε = 0.1.

This behavior is consistent with the ε-soft policy framework described in Chapter 5 of Sutton & Barto, where higher ε promotes exploration at the cost of stability, while lower ε favors exploitation but risks converging prematurely to suboptimal policies.

AI Use Reflection

Initial Interaction

I initially used ChatGPT to generate a baseline implementation of Monte Carlo control for the Blackjack environment, including episode generation and ε-greedy policy logic.

I asked:
"How can I implement Monte Carlo control with ε-greedy policy for the Blackjack environment in Gymnasium?"

The AI suggested generating full episodes, storing state-action-reward sequences, and updating action-value estimates using the average return after each episode.

Iteration 1:

I encountered a NameError when trying to manually define the state variables.

I asked:

"Why am I getting a NameError when defining the state in Blackjack?"

The AI explained that Gymnasium already provides the state through env.reset() and env.step(). I removed the manual state assignment and used the environment's returned state, which resolved the issue.

Iteration 2:

After running the training loop, I thought the code was not working because there was no visible output.

I asked:

"Why is my Monte Carlo algorithm not producing any output?"

The AI clarified that Monte Carlo methods update values internally and do not produce output unless explicitly printed. Based on this, I added print statements and later visualizations to track the evolution of the value function and policy, confirming that learning was occurring.

Iteration 3:

I initially implemented the return calculation incorrectly by summing rewards forward instead of backward through the episode.

I asked:

"How should returns be calculated in Monte Carlo methods?"

The AI suggested iterating backward through the episode to correctly compute cumulative returns. After implementing reversed iteration, the value updates stabilized and produced more consistent results.

Critical Evaluation

While ChatGPT provided useful guidance, some suggestions required careful interpretation.

For example, although the AI correctly suggested computing returns, it did not initially emphasize the importance of iterating backward for efficient computation. My initial forward implementation led to incorrect value updates, which I identified by observing unstable and inconsistent learning behavior.

This highlighted that AI explanations can omit important implementation details, and results must always be validated empirically.

Learning Reflection

Through this process, I developed a deeper understanding of how Monte Carlo methods rely on complete episodes to estimate returns, which explains their high variance compared to Temporal-Difference methods.

The debugging process also reinforced the importance of correctly computing returns, as even small implementation errors can significantly affect learning stability.

Additionally, I learned that ε-greedy policies maintain continuous exploration, which impacts convergence behavior. Overall, I found that AI tools are most effective when used iteratively and critically evaluated against theoretical expectations.

Speaker Notes

- Problem: Learning optimal policy in Blackjack using Monte Carlo methods
- Method: First-visit MC control with ε-soft policy
- Key design: Episode-based learning and backward return computation
- Challenge: Debugging state handling and return calculation
- Result: Learned policy aligns with Blackjack strategy
- Insight: MC methods are simple but high variance and slow
- Connection: Motivates Temporal Difference methods in future labs